中山大學 SUN YAT-SEN UNIVERSITY

国家超级计算广州中心 NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

并行程序设计与算法（实验）

8 -并行多源最短路径搜索

吴迪、刘学正

中山大学 计算机学院

实验概要

实验内容

- 使用OpenMP/Pthreads/MPI实现多源最短路径算法
- 并行程序运行性能分析

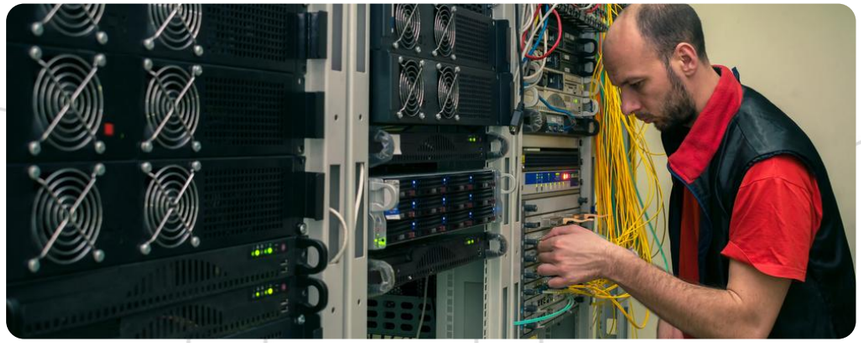
实验目的

- 评估并行化最短路径算法的性能
- 探索不同最短路径算法的并行适应性

背景

- **核心定义：**在图论中，最短路径问题是一个核心的优化问题，旨在寻找图中两个顶点之间总权重最小的路径
- **问题分类**
  - 单源最短路径：Dijkstra算法、Bellman-Ford算法
  - 多源最短路径：Floyd-Warshall算法、Johnson算法
- **典型应用场景：**地图导航 、网络路由 、物流智能调度等





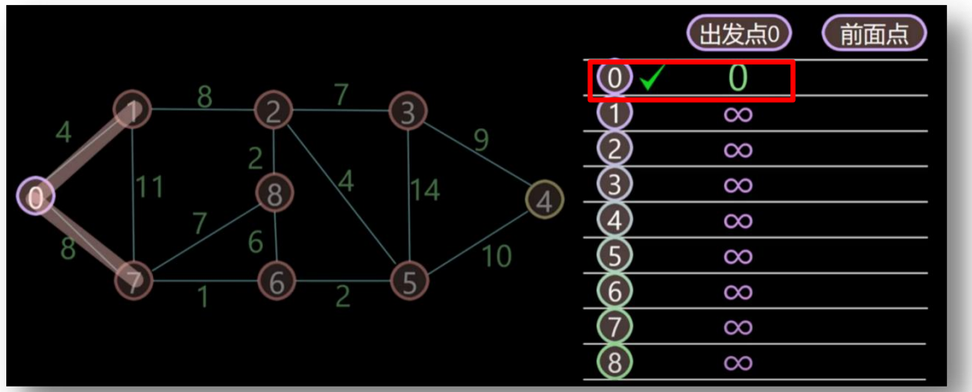


单源最短路径

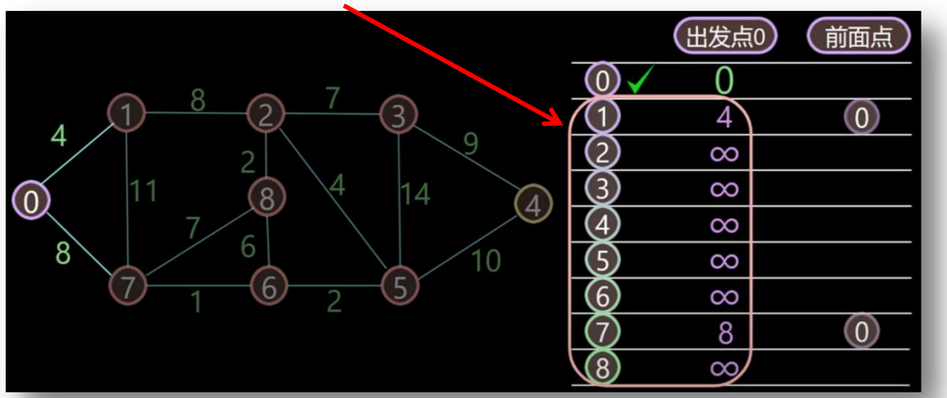
- **Dijkstra 算法：** 每次从未确定最短路径的顶点中，选择距离源点最近的节点并确定其最短路径，再以此点为“跳板”，更新所有邻居节点当前最短路径

松弛操作： $d[v] = \min(d[v], d[u] + w(u, v))$

第0步

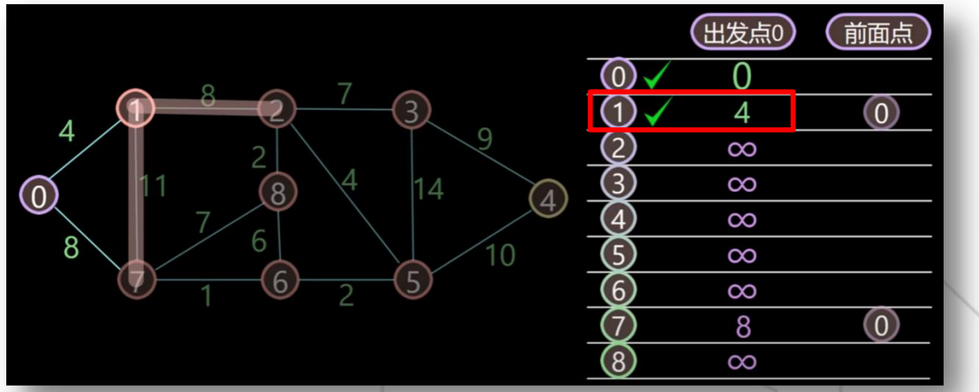


确定点0的最短路径

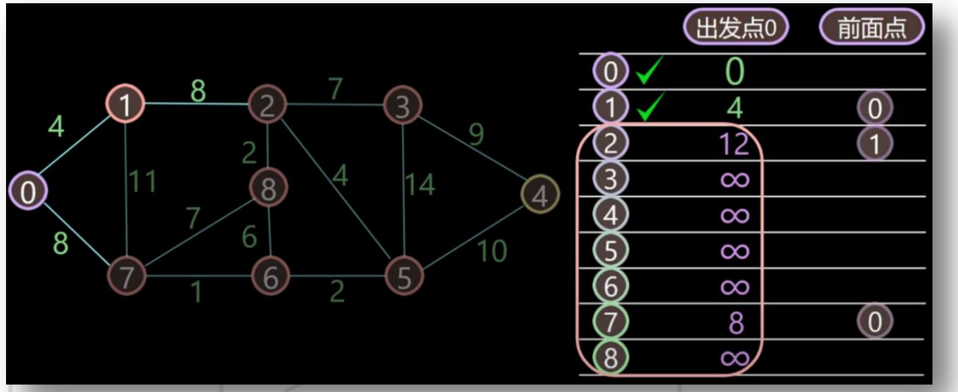


更新点0邻居节点（1、7）当前最短路径

第1步



确定点1的最短路径



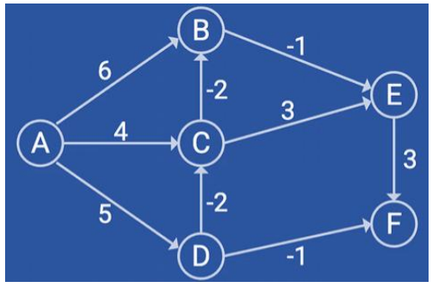
更新点1邻居节点（2、7）当前最短路径

第V-1步 ……

单源最短路径

- **Bellman-Ford算法**：对图中的所有边进行 V-1 次重复的松弛操作，逐步修正最短距离，确保最终得到所有节点之间的最短路径

边遍历顺序示例：A->B, A->C, A->D, B->E, C->B, C->E, D->C, D->F, E->F
可以按照任意顺序遍历边，只会影响收敛速度，不影响结果正确性



带负权的图

顶点	距离	前驱
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-

第0轮

顶点	距离	前驱
A	0	-
B	2	C
C	3	D
D	5	A
E	5	B
F	4	D

第1轮

顶点	距离	前驱
A	0	-
B	2	C
C	3	D
D	5	A
E	5	B
F	4	D

第2轮

.....

第V-1轮松弛
(最多)

松弛操作：$d[v] = \min(d[v], d[u] + w(u, v))$

多源最短路径

- **Floyd-Warshall 算法：** 任意两点间的最短路径要么是直接相连的边，要么经过若干个其他顶点作为中转。算法通过动态规划，逐个引入每个顶点作为允许的中转点，不断更新所有点对之间的最短路径。

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	∞
B	5	0	∞	1	6	∞	∞
C	2	∞	0	6	∞	8	∞
D	∞	1	6	0	1	2	∞
E	∞	6	∞	1	0	∞	7
F	∞	∞	8	2	∞	0	3
G	∞	∞	∞	∞	7	3	0

第0步：不经过中间节点（邻接矩阵初始化）

	A	B	C	D	E	F	G
A	0	5	2	∞	∞	∞	∞
B	5	0	7	1	6	∞	∞
C	2	7	0	6	∞	8	∞
D	∞	1	6	0	1	2	∞
E	∞	6	∞	1	0	∞	7
F	∞	∞	8	2	∞	0	3
G	∞	∞	∞	∞	7	3	0

第1步：以A为中间节点

	A	B	C	D	E	F	G
A	0	5	2	6	11	∞	∞
B	5	0	7	1	6	∞	∞
C	2	7	0	6	∞	8	∞
D	6	1	6	0	1	2	∞
E	11	6	∞	1	0	∞	7
F	∞	∞	8	2	∞	0	3
G	∞	∞	∞	∞	7	3	0

第2步：以B为中间继节点

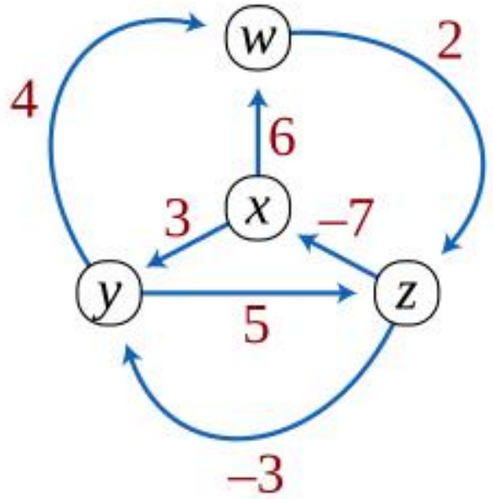
第V步：…

```
d[B][C] = min(d[B][C], d[B][A]+d[A][C]) = 7
```

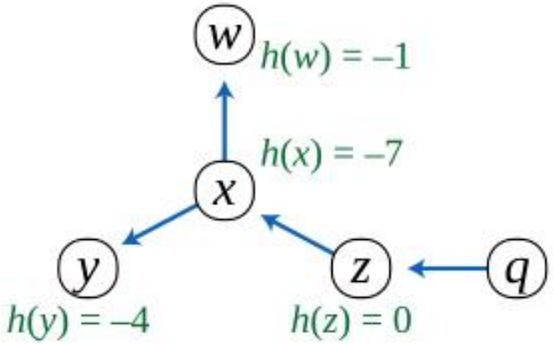
多源最短路径

- **Johnson算法：** 首先利用 Bellman-Ford 计算势能进行重赋权，将负权边转换为非负权边；随后对每个顶点运行 Dijkstra 算法，高效求解所有点对的最短路径

节点间的权值： $w(u,v)$



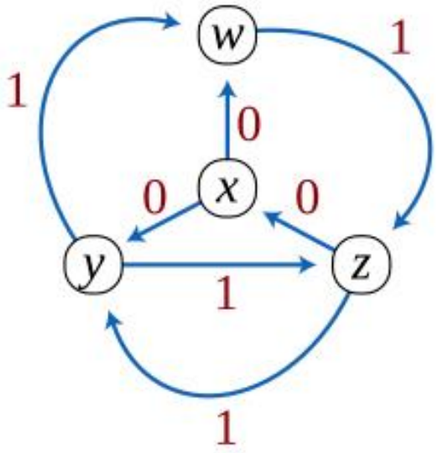
带负权边的原始图



添加虚拟点q（到其余各点的距离为0），运行Bellman-Ford算法计算点q到其余点的最短路径h

节点间的调整后的权值：

$$w'(u,v) = w(u,v) + h[u] - h[v]$$



调整边权后的正权边图

运行多次Dijkstra算法

算法对比

Dijkstra

单源算法　❌不支持负权边

📌**适用场景**：无负权边的单源最短路径求解

⏱**时间复杂度**：

朴素版 $O(V^2)$

堆优化版 $O((V+E)\log V)$

✨**核心优势**：综合效率极高，是无负权图的标准解法

Bellman-Ford

单源算法　☑支持负权边

📌**适用场景**：图中存在负权边，且需检测是否存在负权环

⏱**时间复杂度**：

$O(V*E)$

✨**核心优势**：不仅支持负权边，还能检测环路

Floyd-Warshall

多源算法　☑支持负权边

📌**适用场景**：稠密图（顶点数适中）的多源最短路径求解，边权可为负，但不能存在负权环

⏱**时间复杂度**：

$O(V^3)$

✨**核心优势**：实现极简，一次性计算所有点对距离

Johnson

多源算法　☑支持负权边

📌**适用场景**：顶点数较多的稀疏图场景，边权可为负，可处理负权环

⏱**时间复杂度**：

$O(V*E + V^2\log V)$

✨**核心优势**：融合了Bellman-Ford和Dijkstra的优点，适合处理稀疏图

并行多源最短路径算法

背景与问题

- 传统串行算法求解所有点对最短路径，要么逐个源点计算，要么整体计算但复杂度过高，在大规模图上效率低下
- 能否让多个计算单元并行协同，一次性得到所有点对的最短路径?

核心目标

- 性能加速：相比最优串行实现，达到明显的时间收益
- 结果等价：保持与原串行算法等价的正确性
- 可扩展：控制通信与同步成本，提升并行效率

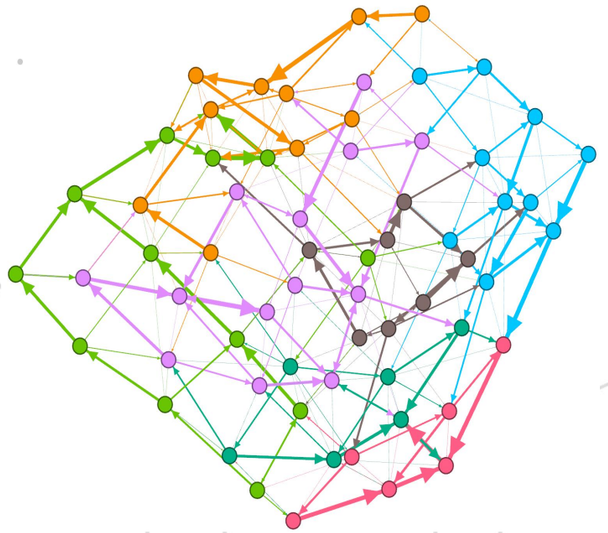
核心思想

- 识别并行执行算法中的独立计算单元（如边松弛、矩阵分块、源点分组），同时控制通信与同步开销

实验内容

- **并行多源最短路径搜索**
  - 使用OpenMP/Pthreads/MPI中的任意一种
  - 可选用任意最短路径算法
  - 设置线程数量（1-16）
  - 根据运行时间，分析程序并行性能
    - 算法、并行框架、并行方式、数据（节点数量，平均度数）

Source	Target	Weight
0	1	0.7214398
0	2	0.0547731
0	4	0.2237872
1	5	0.2278107
1	2	0.5384615
1	6	0.193787
1	7	0.010355
1	8	0.0295858
5	7	0.8267477
5	9	0.1489362
5	10	0.0243161
7	10	0.557377
7	11	0.4036885
7	12	0.0245902
7	13	0.0061475



Questions?

